INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Plataformas Emergentes				
TÍTULO DE LA PRÁCTICA:	*Framework Multiplataforma*				
NÚMERO DE PRÁCTICA:	*04*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*X*
FECHA DE PRESENTACIÓN	*11/10/2026*	**HORA DE PRESENTACION**	*23:00 pm*		
INTEGRANTE (s): - Carrasco Choque, Arles Melvin - Condori Leon, Joel Isaias - Yana Agramonte, Luis Efrain				**NOTA:**	
DOCENTE(s): *M. Sc. Ing. Rene Alonso Nieto Valencia*					

SOLUCIÓN Y RESULTADOS

LINK DEL REPOSITORIO:
https://github.com/YoelLeon/PE-2026B-labs/tree/main/LAB04-Plataformas_Emergentes/misti_verde

1. DESCRIPCIÓN GENERAL DEL PROGRAMA

1.1 Nombre y propósito

MistiVerde es una aplicación móvil multiplataforma, desarrollada con **Flutter**, que orienta a los habitantes de Arequipa sobre cómo y dónde reciclar. El nombre une el volcán Misti, símbolo de la ciudad, con lo ecológico. La aplicación permite clasificar residuos según el código de colores de tachos de la NTP 900.058:2019 (ámbito municipal), consultar puntos de acopio en lista y en mapa, registrar el reciclaje realizado y ganar “hojas” (10 hojas por kg) que hacen subir de nivel al usuario cada 100 hojas: Semilla, Brote, Arbusto, Árbol y Guardián del Misti.

Es un prototipo académico del Laboratorio 4 de Plataformas Emergentes, centrado en el diseño de interfaces con los componentes propios del framework. Todo el código está en un único archivo

(lib/main.dart), no usa servidor ni base de datos y guarda los datos en el propio equipo con shared_preferences. Usa Material Design 3, una paleta propia (verde Misti #2E7D32, verde oscuro #1B5E20 y blanco “sillar” #F7F4EC) y modo claro y oscuro automático.

1.2 Problema y árbol de problemas

Problema abordado: los habitantes de Arequipa tienen dificultad para reciclar correctamente porque no saben cómo clasificar sus residuos, dónde llevarlos ni cuentan con un incentivo para hacerlo de forma constante. La Tabla 1 resume el árbol de problemas en el que se basan las funcionalidades y las interfaces.

Tabla 1. Árbol de problemas del proyecto

Nivel	Contenido
Efectos	• Mayor contaminación y acumulación de residuos. • Residuos reciclables que terminan en la basura común. • Menor vida útil de los rellenos sanitarios.
Problema central	Bajo nivel de reciclaje adecuado en la población arequipeña.
Causas	• Desconocimiento de cómo clasificar los residuos. • Poca información sobre puntos de acopio. • Falta de hábito y de motivación. • Ausencia de herramientas digitales de seguimiento.

Cada causa se atiende con una funcionalidad: el clasificador y las categorías (desconocimiento), la lista y el mapa de puntos de acopio (poca información), los retos semanales, las hojas y los niveles (hábito y motivación) y el registro con historial e impacto (seguimiento).

1.3 Objetivo y usuarios

Objetivo general: desarrollar una aplicación móvil multiplataforma que oriente a los usuarios sobre cómo y dónde reciclar y los motive con un sistema de puntos. **Usuario principal:** jóvenes y estudiantes. **Usuario secundario:** familias y personas interesadas en reciclar.

1.4 Antecedentes

Ya existe la aplicación oficial **Arequipa Recicla**, de la Municipalidad Provincial de Arequipa, que ofrece guía de reciclaje, consejos, recompensas por puntos y avisos de fechas y horarios de recojo en hogares. MistiVerde no la reemplaza ni gestiona el recojo domiciliario: es un prototipo académico centrado en el diseño de interfaces con Flutter, orientado a practicar la clasificación y al seguimiento personal del reciclaje. La propia aplicación remite a Arequipa Recicla cuando no hay puntos de acopio verificados para una categoría.

1.5 Alcance y limitaciones

Incluye: diseño de interfaces y navegación, lógica de hojas, niveles y retos semanales, persistencia local, perfil local (onboarding y formulario validado) y un mapa con OpenStreetMap en la pestaña Puntos.

No incluye y debe tenerse en cuenta:

- El perfil es local: no hay contraseña, servidor ni autenticación real.
- No usa GPS ni la ubicación del usuario. Los marcadores del mapa son una referencia aproximada del centro del distrito, no la dirección exacta.
- El registro de kilogramos es autodeclarado: la aplicación no lo verifica.
- Los puntos de acopio provienen de fuentes públicas, algunas con varios años de antigüedad, y no incluyen horarios porque no pudieron verificarse. Deben confirmarse antes de ir.
- Los datos locales funcionan sin conexión, pero las teselas del mapa requieren internet. No se realizó una medición formal de rendimiento.

1.6 Consideraciones éticas, sociales, legales y económicas

- **Ética y social:** promueve la conciencia ambiental, no recolecta datos personales más allá de un nombre, un alias y un distrito que se guardan solo en el equipo, y el usuario lo acepta de forma explícita en el formulario.
- **Legal:** no maneja información sensible; usa la NTP 900.058:2019 como referencia de colores y cita la fuente de cada punto de acopio. Los datos del mapa pertenecen a los colaboradores de OpenStreetMap y se muestran con su atribución.
- **Económico:** la aplicación es gratuita y el framework es de código abierto.

2. DISEÑO DE INTERFACES

2.1 Funcionalidades (requerimientos funcionales)

Las funcionalidades se derivan de las causas del árbol de problemas y se resumen en la Tabla 2.

Tabla 2. Requerimientos funcionales

Cód.	Requerimiento	Pantalla
RF01	Mostrar el resumen de hojas del usuario y un consejo del día.	Inicio
RF02	Mostrar las 8 categorías de residuos (Plástico, Papel y cartón, Vidrio, Metal, Orgánico, Electrónicos, Peligrosos y No aprovechable).	Inicio
RF03	Buscar un residuo por nombre y filtrar por categoría.	¿Dónde lo tiro?
RF04	Indicar el tacho (NTP 900.058:2019) y la instrucción de preparación de cada residuo.	¿Dónde lo tiro?
RF05	Listar puntos de acopio con ubicación, materiales que acepta y fuente de la información.	Puntos
RF06	Filtrar los puntos de acopio por tipo de material.	Puntos
RF07	Registrar reciclaje (categoría reciclable y kg). Peligrosos y No aprovechable no se registran.	Panel Registrar
RF08	Calcular y acumular hojas (10 por kg).	Panel Registrar, Impacto
RF09	Mostrar el nivel temático y el progreso al siguiente nivel.	Impacto
RF10	Mostrar el total de kg reciclados y el historial con fecha.	Impacto

RF11	Mostrar retos semanales con barra de progreso (se calculan desde el lunes de la semana actual).	Inicio
RF12	Mostrar el detalle de cada categoría, con advertencia si no es reciclable y dónde llevarlo.	Detalle de categoría
RF13	Mostrar el detalle de un punto de acopio con el botón “Copiar ubicación”.	Detalle de punto
RF14	Guardar el historial en el equipo y permitir reiniciar los datos.	Impacto
RF15	Mostrar el resumen por categoría (tabla con kg y proporción).	Impacto
RF16	Presentar la app (onboarding), crear y editar un perfil local validado y cerrar sesión.	Onboarding, Perfil, Menú lateral
RF17	Mostrar los puntos de acopio con ubicación en un mapa real.	Puntos (pestaña Mapa)
RF18	Eliminar un registro del historial deslizando, con opción de deshacer.	Impacto

2.2 Requerimientos no funcionales

Tabla 3. Requerimientos no funcionales

Cód.	Requerimiento	Cómo se atiende
RNF01	Usabilidad	Barra de navegación inferior de 4 secciones, menú lateral y botón flotante “Registrar”.
RNF02	Portabilidad	Un solo código base de Flutter para Android, iOS, web y escritorio (en las pruebas se ejecutó en Chrome).
RNF03	Rendimiento	Respuesta inmediata en las pruebas manuales; no se realizó una medición formal.
RNF04	Diseño	Material Design 3, identidad visual temática (Misti y sillar) y modo oscuro automático.
RNF05	Mantenibilidad	Widgets reutilizables y código comentado, con el historial de cambios marcado en comentarios.
RNF06	Responsividad	Ancho máximo de 640, LayoutBuilder (3 o 4 columnas), MediaQuery, AspectRatio, Expanded, Flexible, Spacer y Wrap.
RNF07	Funcionamiento sin conexión	Datos locales sin internet; solo las teselas del mapa requieren conexión.

2.3 Estructura y navegación

Al abrir la app, la pantalla de bienvenida lee el perfil guardado. Si no existe (primera vez) lleva al onboarding y al formulario de perfil; si existe, entra directamente a la pantalla principal (HomeShell), que contiene la barra de navegación inferior con cuatro pestañas, el menú lateral y el botón flotante “Registrar”. Desde Inicio se abre el detalle de una categoría y desde Puntos el detalle de un punto de acopio.

Tabla 4. Pantallas de la aplicación

Pantalla	Clase en el código	Función
Bienvenida	SplashPage	Animación inicial y decisión de a dónde ir.
Onboarding	OnboardingPage	Tres pantallas de presentación (PageView).
Crear / editar perfil	PerfilFormPage	Formulario validado con nombre, alias, distrito y consentimiento.
Contenedor principal	HomeShell	Estado compartido, barra inferior, menú lateral y botón Registrar.
Inicio	InicioPage	Banner, consejo, retos semanales y categorías.
Detalle de categoría	CategoriaPage	Residuos de la categoría y dónde llevarlos.
¿Dónde lo tiro?	ClasificadorPage	Búsqueda y filtro de residuos con su tacho.
Puntos	PuntosPage, _MapaPuntos	Lista y mapa de puntos de acopio.
Detalle de punto	PuntoPage	Ficha del punto con fuente y aviso.
Impacto	ImpactoPage	Nivel, estadísticas, resumen por categoría e historial.

2.4 Mockups

Los mockups siguientes son wireframes de baja fidelidad que se elaboraron antes de programar. Muestran la organización de cada interfaz y los widgets de layout previstos. Las capturas de la aplicación terminada se presentan en la sección 3.3.

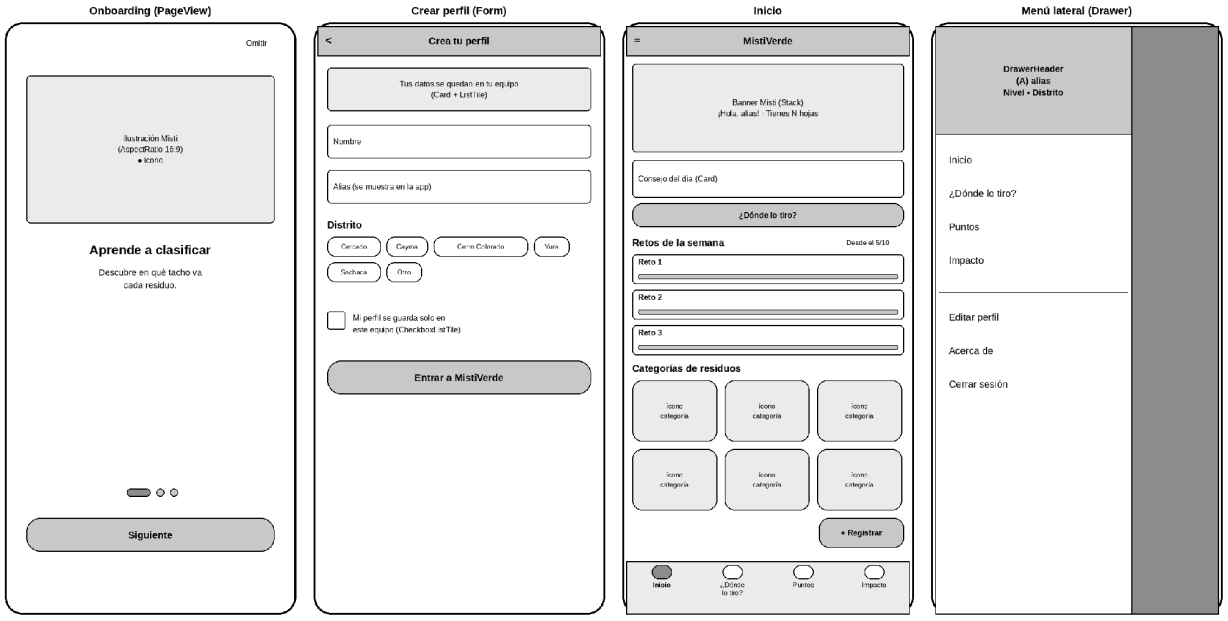


Figura 1. Mockup 1: onboarding, creación de perfil, inicio y menú lateral

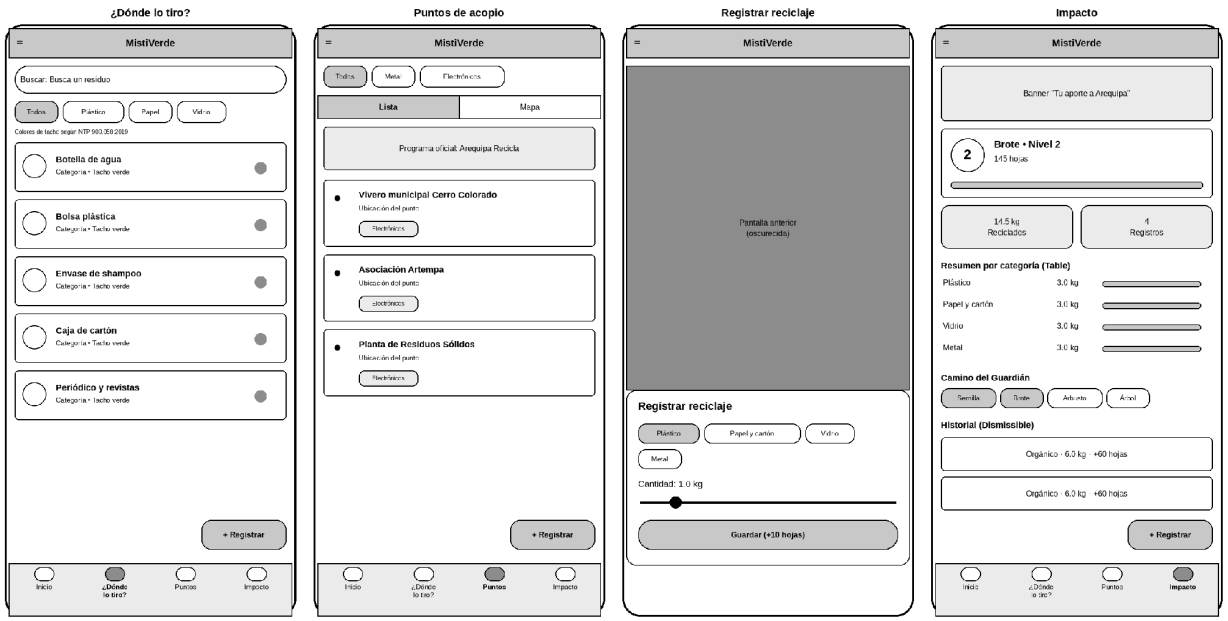


Figura 2. Mockup 2: clasificador, puntos de acopio, registro de reciclaje e impacto

3. DESCRIPCIÓN DEL PROCESO DE DESARROLLO

3.1 Entorno y configuración

El proyecto se desarrolló en Visual Studio Code con el SDK de Flutter instalado en el equipo. Se creó el proyecto misti_verde, se agregaron los paquetes necesarios, se declaró la carpeta de imágenes en el archivo pubspec.yaml y se ejecutó la aplicación en Chrome con flutter run (opción Chrome). La Figura 3 muestra el editor con el archivo main.dart y la terminal con los dispositivos disponibles.

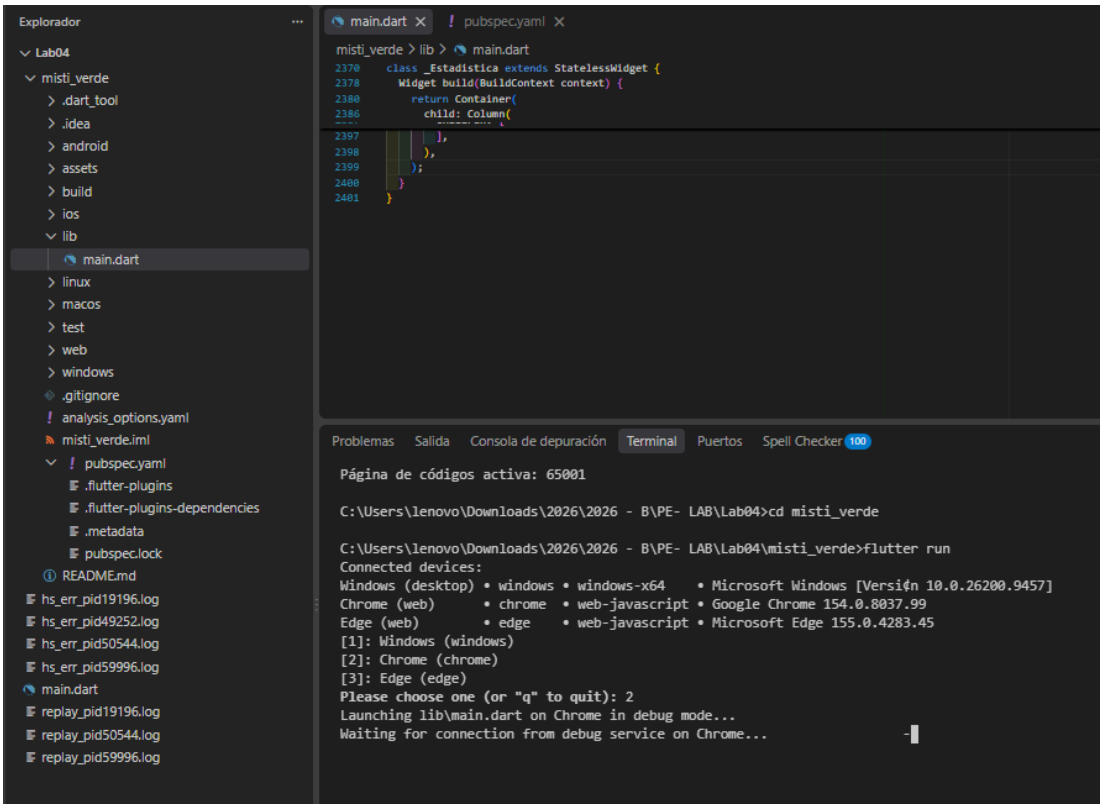


Figura 3. Entorno de desarrollo: Visual Studio Code y ejecución con flutter run

Pasos de configuración:

- Crear el proyecto: flutter create misti_verde.
- Agregar paquetes: flutter pub add shared_preferences flutter_map latlong2.
- Declarar las imágenes en pubspec.yaml (assets: - assets/images/) y ejecutar flutter pub get.
- Reemplazar el contenido de lib/main.dart por el código del Anexo A y ejecutar flutter run. Tras agregar assets hay que reiniciar la aplicación completa, porque la recarga rápida no los carga.
- Para ver la vista móvil en Chrome: F12 y Ctrl+Shift+M.

3.2 Estructura del código

Todo el código está en lib/main.dart y se organiza en este orden: paleta de colores, modelos de datos, datos constantes, funciones auxiliares, ilustración propia, aplicación principal y pantallas. Los datos incluyen 8 categorías, 25 residuos, 5 puntos de acopio, 3 retos, 6 consejos y 18 distritos. Cada registro

de reciclaje se guarda como texto con el formato “categoría|kg|hojas|fecha ISO”. Los paquetes usados se listan en la Tabla 5.

Tabla 5. Paquetes utilizados

Paquete	Uso
flutter/material.dart	Widgets de Material Design, tema, navegación y layout.
flutter/services.dart	Portapapeles (Clipboard) y formateadores de texto.
shared_preferences	Guardar el perfil y el historial de registros en el equipo.
flutter_map	Mapa con teselas de OpenStreetMap y marcadores.
latlong2	Coordenadas (LatLng) para el mapa.

Modelos principales: Categoria, Contenedor (tacho según la NTP), Residuo, PuntoAcopio (con fuente y coordenadas opcionales), Registro (con fecha), Perfil (con carga, guardado y borrado local) y Reto.

3.3 Pantallas desarrolladas y widgets utilizados

A continuación se muestra cada pantalla de la aplicación terminada y se explica cómo funcionan los widgets que la construyen.

3.3.1 Pantalla de bienvenida (Splash)

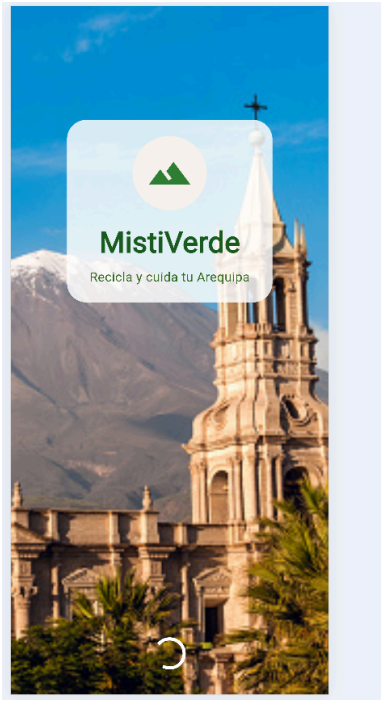


Figura 4. Pantalla de bienvenida

Muestra el nombre y el lema sobre la ilustración del Misti mientras la app lee el perfil guardado. Después pasa al onboarding (primera vez) o a la pantalla principal.

- **Stack** (fit: expand): superpone ilustración, foto, tarjeta y barra de progreso.
- **CustomPaint** y **MistiPainter**: dibujan con Canvas el cielo, el Misti y la ciudad. Sirven de respaldo si falta la foto.
- **Image.asset** con errorBuilder: carga la foto real y no se rompe si el archivo no existe.
- **Align**: ubica la tarjeta y el indicador de carga con coordenadas relativas.
- **TweenAnimationBuilder**: anima opacidad y escala de la tarjeta en 900 ms.
- **PageRouteBuilder** con **FadeTransition**: transición con desvanecimiento hacia la siguiente pantalla.

3.3.2 Onboarding

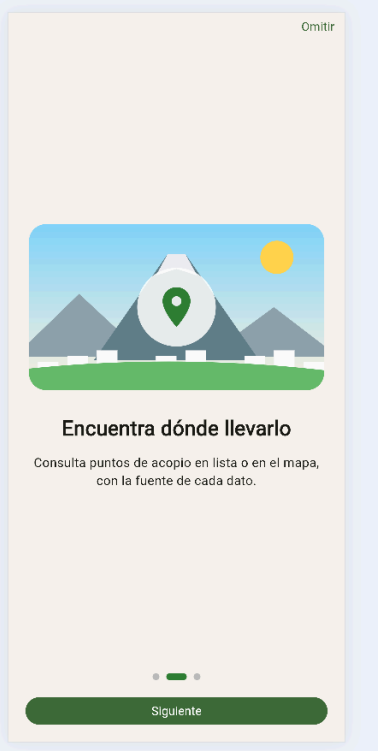


Figura 5. Una pantalla del onboarding con los puntos indicadores

Tres pantallas de presentación (clasificar, encontrar dónde llevar y ganar hojas). El botón cambia a “Crear mi perfil” en la última.

- **PageView** con **PageController**: páginas deslizables; nextPage avanza con animación.
- **AnimatedContainer**: el indicador de página crece y cambia de color suavemente.
- **AspectRatio** (16/9): mantiene la proporción de la ilustración.
- **ConstrainedBox** (maxWidth 640) dentro de **Center**: limita el ancho en pantallas grandes.
- **TextButton** (Omitir) y **FilledButton** (Siguiente): navegación entre pasos.

3.3.3 Formulario de perfil: validación

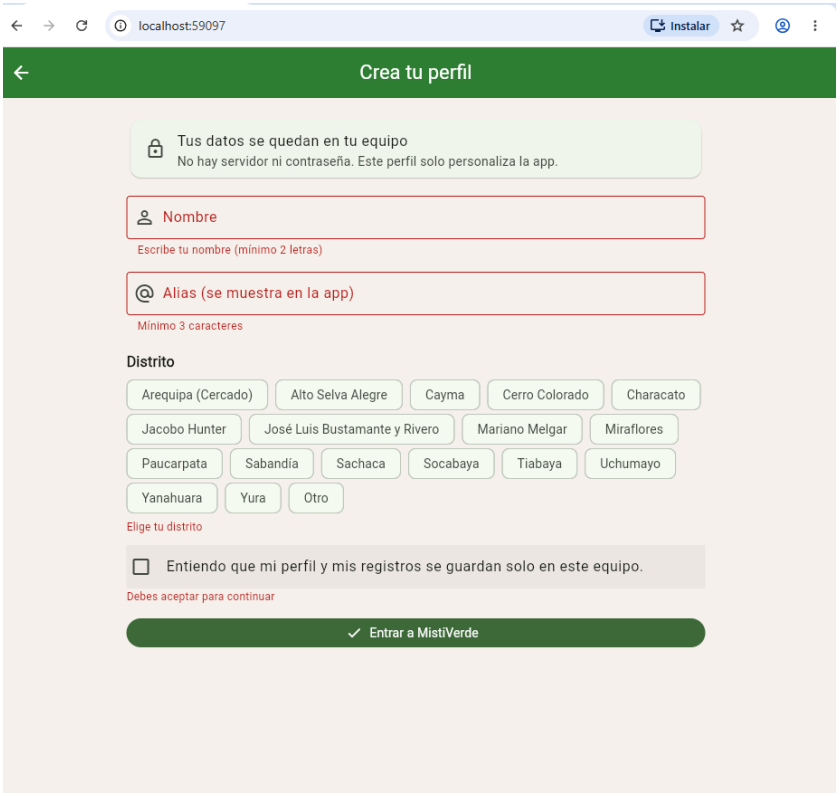


Figura 6. Formulario enviado vacío, con los errores en rojo

El formulario valida los datos al pulsar el botón y muestra los errores en rojo bajo cada campo, sin guardar nada hasta que todo sea correcto.

- **Form** con GlobalKey<FormState>: agrupa los campos y ejecuta validate().
- **TextFormField** con validator: el nombre exige 2 letras como mínimo y el alias 3 caracteres (letras, números y guion bajo).
- **LengthLimitingTextInputFormatter**: limita el alias a 15 caracteres.
- **FormField<String>** y **FormField<bool>**: validan el distrito y el consentimiento, que no son campos de texto.

3.3.4 Formulario de perfil: datos completos



Figura 7. Formulario lleno, con distrito y casilla marcados

Con los datos correctos, el botón guarda el perfil en el equipo y abre la pantalla principal. El mismo formulario sirve para editar el perfil.

- **ChoiceChip** dentro de **Wrap**: los distritos se acomodan en varias líneas y se elige uno.
- **CheckboxListTile**: consentimiento de que los datos se guardan solo en el equipo.
- **FilledButton.icon**: guarda con Perfil.guardar() (shared_preferences) y navega con pushAndRemoveUntil.
- **SingleChildScrollView**: permite desplazar el formulario en pantallas pequeñas.

3.3.5 Inicio (parte superior)

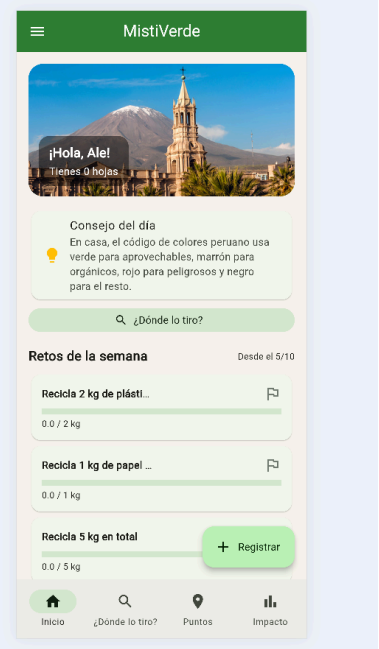


Figura 8. Inicio con el banner, "¡Hola, alias!" y el consejo

Pantalla principal: saluda con el alias, muestra las hojas acumuladas y un consejo del día elegido según el día del mes.

- **ListView**: columna desplazable con todo el contenido.
- **MistiBanner** (ClipRRect, SizedBox y **Stack**): ilustración y foto con bordes redondeados.
- **Positioned** y **Container**: cuadro semitransparente con el saludo en la esquina inferior.
- **Card** con **ListTile**: consejo del día.
- **MediaQuery**: la altura del banner cambia con la orientación y el ancho (120, 180 o 220).
- **Row** con **Expanded** y **FilledButton.tonalIcon**: acceso rápido al clasificador.

3.3.6 Inicio (retos y categorías)

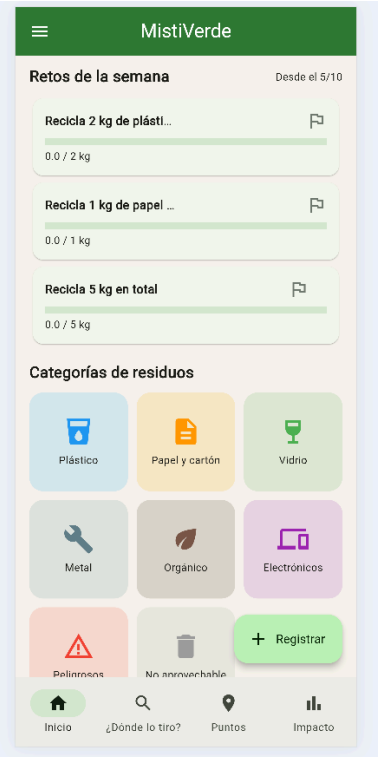


Figura 9. Inicio desplazado: retos y cuadrícula de categorías

Los retos semanales se calculan con los registros desde el lunes de la semana actual. Debajo aparecen las 8 categorías.

- **Row** y **Spacer**: el título queda a la izquierda y la fecha de inicio a la derecha.
- **Flexible** con ellipsis: evita que el título del reto desborde.
- **LinearProgressIndicator** dentro de **TweenAnimationBuilder**: barra de avance animada.
- **LayoutBuilder**: usa 3 o 4 columnas según el ancho disponible.
- **GridView.count** (shrinkWrap y NeverScrollableScrollPhysics): cuadrícula dentro del ListView.
- **Material**, **InkWell** y **Hero**: cada categoría es tocable y anima su ícono hacia el detalle.

3.3.7 Menú lateral

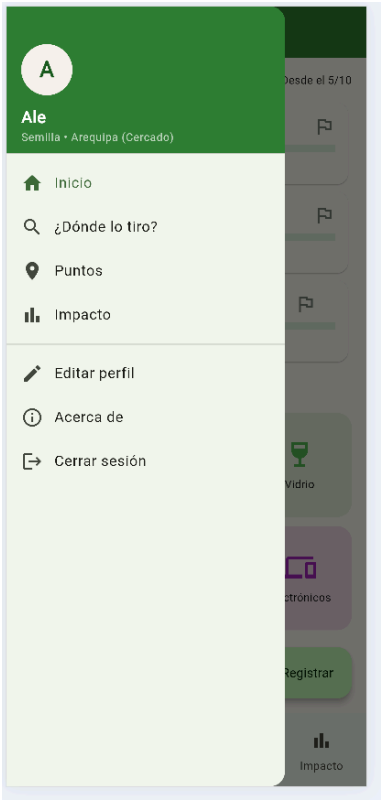


Figura 10. Menú lateral abierto

Muestra el perfil del usuario, atajos a las cuatro secciones y las opciones de editar perfil, acerca de y cerrar sesión.

- **Drawer** con **ListView**: menú lateral desplazable.
- **DrawerHeader** con **CircleAvatar**: inicial del alias, nivel y distrito.
- **ListTile** (selected): opciones con ícono; la sección activa se resalta.
- **showAboutDialog** y **AlertDialog**: información de la app y confirmación al cerrar sesión (borra el perfil local).

3.3.8 Detalle de categoría (encabezado expandido)



Figura 11. Detalle de "Plástico" con el encabezado grande

Lista los residuos de la categoría con su tacho e instrucción, y los puntos donde llevarlos.

- **CustomScrollView** con **SliverAppBar** (pinned, expandedHeight 200): encabezado que se colapsa al desplazar.
- **FlexibleSpaceBar** con **Stack**: color de fondo, foto, **DecoratedBox** con degradado e ícono.
- **Hero**: el ícono viaja desde la cuadrícula de Inicio.
- **SliverPadding** y **SliverList**: contenido desplazable bajo el encabezado.
- **Card** con **ListTile** (isThreeLine): tacho e instrucción de cada residuo.

3.3.9 Detalle de categoría (encabezado colapsado)

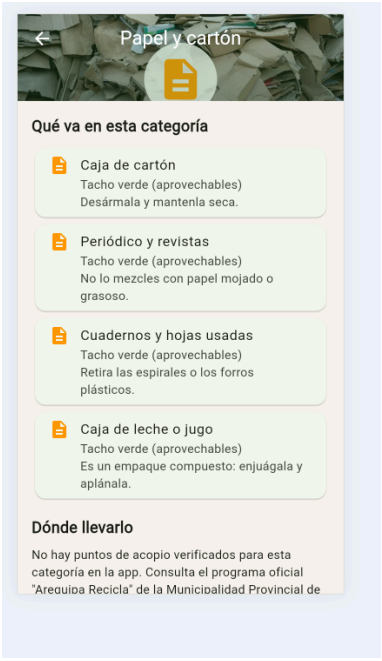


Figura 12. La misma pantalla ya desplazada

Al desplazar, el encabezado grande se reduce a una barra normal que conserva el título y el botón de volver.

- **SliverAppBar** con pinned: true: la barra permanece visible al terminar de colapsar.
- **Wrap** con **Chip**: puntos de acopio donde llevar la categoría, acomodados en varias líneas.
- **Center** y **ConstrainedBox**: mantienen el ancho máximo de 640.

3.3.10 Categoría no reciclable

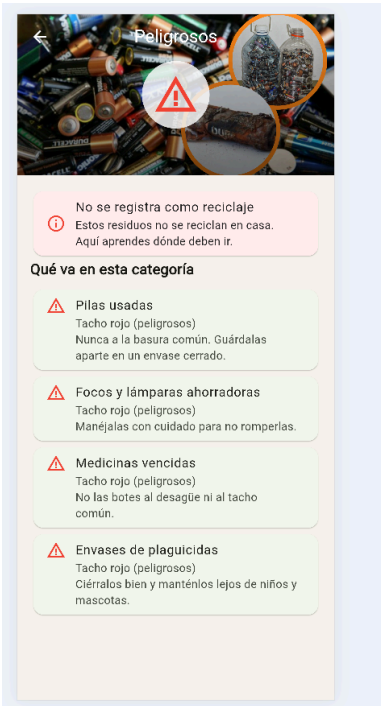


Figura 13. Categoría Peligrosos con el aviso de no reciclable

Peligrosos y No aprovechable no se registran como reciclaje. La pantalla lo advierte y no ofrece el bloque “Dónde llevarlo”.

- **Renderizado condicional** (if con categoria.reciclable): la tarjeta de aviso y el bloque de puntos aparecen solo cuando corresponde.
- **Card** con color shade50 y **ListTile**: aviso rojo con ícono de información.
- **Residuo.contenedor**: cada residuo indica el tacho rojo o negro de la NTP.

3.3.11 ¿Dónde lo tiro? (clasificador)



Figura 14. Lista de residuos con la barra de búsqueda y los filtros

Permite buscar un residuo y filtrar por categoría. Cada fila muestra el tacho recomendado con un punto de color.

- **Column** con **Expanded**: la lista ocupa el espacio restante bajo la búsqueda y los filtros.
- **TextField** con OutlineInputBorder y prefixIcon: barra de búsqueda redondeada.
- **ListView** horizontal con **ChoiceChip**: filtro por categoría.
- **ListView.builder**: crea solo las filas visibles de la lista.
- **ListTile** con **CircleAvatar** y **Container** circular: ícono de la categoría y color del tacho.

3.3.12 Clasificador: búsqueda en vivo

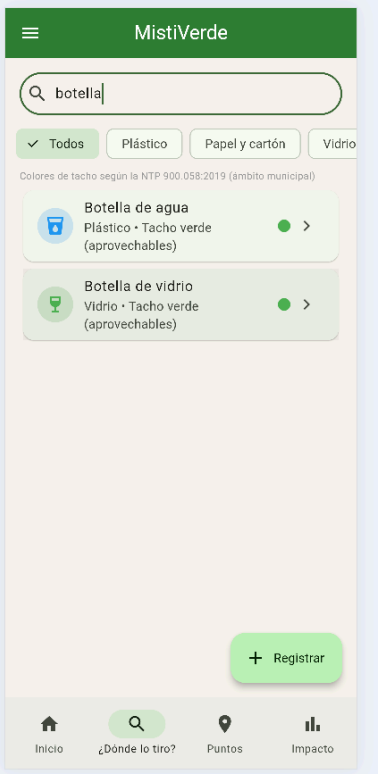


Figura 15. Búsqueda de "botella" con el filtro aplicado

La lista se actualiza mientras se escribe y combina el texto buscado con el filtro de categoría.

- **setState** con onChanged: recalcula la lista filtrada en cada cambio.
- Filtro con **where** sobre nombre (contains) y categoría.
- Estado vacío con **Center**, **Column** e ícono cuando no hay resultados.

3.3.13 Clasificador: diálogo del residuo

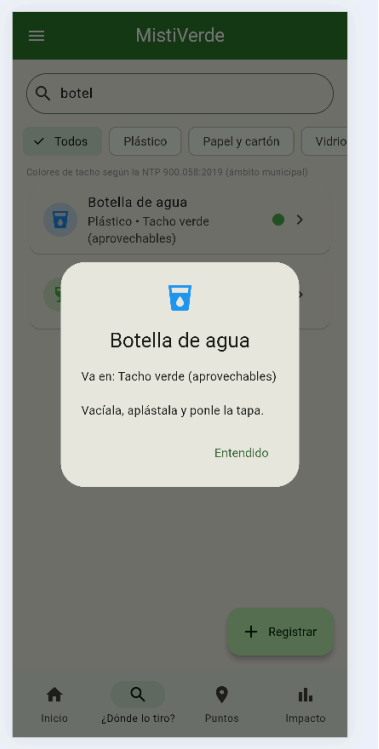


Figura 16. Diálogo con el tacho y la instrucción de un residuo

Al tocar un residuo se abre un diálogo con el tacho donde va y la instrucción para prepararlo.

- **showDialog** con **AlertDialog**: ventana modal con ícono, título, contenido y acción.
- **TextButton** (Entendido) con Navigator.pop: cierra el diálogo.

3.3.14 Puntos de acopio: lista

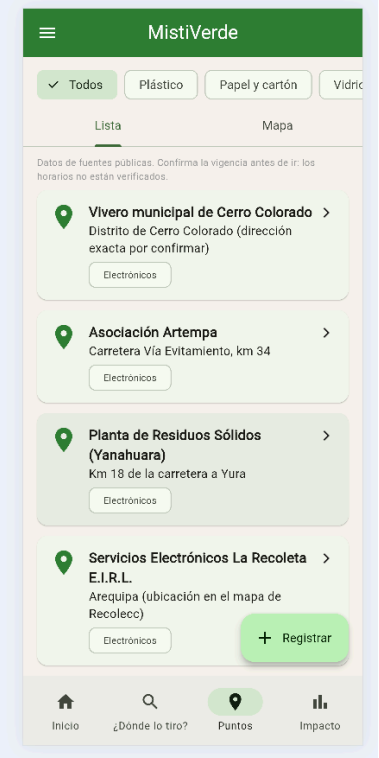


Figura 17. Pestaña Lista de Puntos

Muestra los puntos con su ubicación y los materiales que aceptan. El filtro superior se comparte con la pestaña Mapa.

- **DefaultTabController**, **TabBar** y **TabBarView**: pestañas Lista y Mapa (sin deslizar, para no chocar con el gesto del mapa).
- **ChoiceChip** en **ListView** horizontal: filtro por material.
- **Card** (clipBehavior) con **InkWell**: ficha tocable que abre el detalle.
- **Row**, **Expanded** y **Wrap** con **Chip**: ícono, textos y materiales aceptados.
- **Hero**: el ícono del punto viaja hacia la pantalla de detalle.

3.3.15 Puntos de acopio: mapa

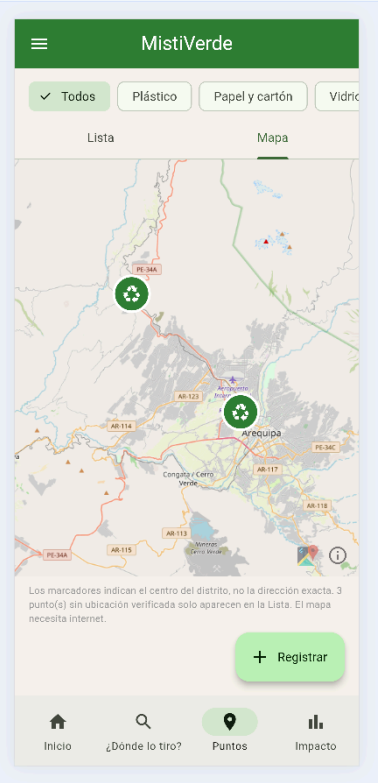


Figura 18. Pestaña Mapa con los marcadores y las teselas cargadas

Mapa real con teselas de OpenStreetMap. Solo se dibujan los puntos con coordenadas, y los marcadores son una referencia del distrito, no la dirección exacta.

- **FlutterMap** con MapOptions: centro en Arequipa, zoom inicial 10.5 y límites de zoom.
- **TileLayer**: descarga las teselas de OpenStreetMap (requiere internet).
- **MarkerLayer** y **Marker** con **GestureDetector**: marcadores tocables que abren el detalle.
- **RichAttributionWidget**: muestra la atribución “OpenStreetMap contributors”.
- **Column** con **Expanded**: el mapa ocupa el alto disponible y una nota aclara sus límites.

3.3.16 Detalle de un punto de acopio

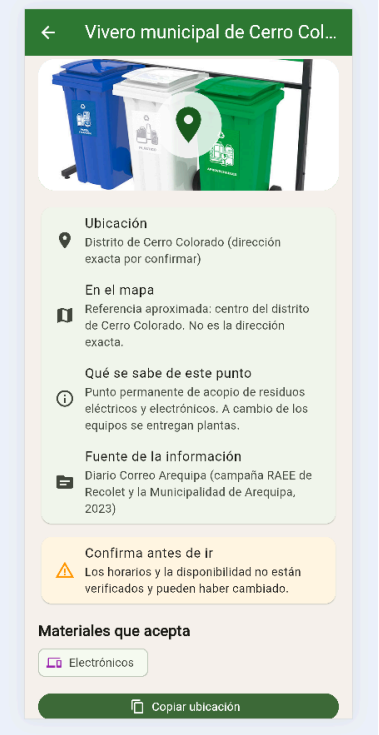


Figura 19. Ficha de un punto con la fuente y el aviso

Presenta ubicación, referencia en el mapa, lo que se sabe del punto y la fuente de la información, con un aviso para confirmar antes de ir.

- **AspectRatio** (16/7): el banner mantiene su proporción.
- **Card** con **Column** de **ListTile**: ubicación, mapa, detalle y fuente.
- **Card** ámbar: aviso “Confirma antes de ir”.
- **Wrap** con **Chip**: materiales que acepta.
- **Clipboard.setData** y **SnackBar**: el botón “Copiar ubicación” copia el texto y confirma.

3.3.17 Registrar reciclaje

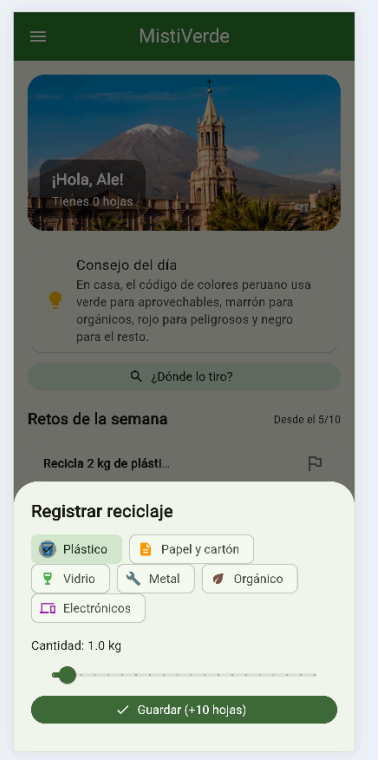


Figura 20. Panel "Registrar reciclaje" abierto

Panel inferior para registrar lo reciclado. Solo ofrece categorías reciclables y calcula las hojas (10 por kg).

- **showModalBottomSheet** (isScrollControlled): panel que sube desde abajo.
- **StatefulBuilder**: actualiza solo el panel al cambiar la categoría o los kg.
- **ChoiceChip** en **Wrap**: elección de categoría.
- **Slider** (0.5 a 10 kg, 19 divisiones): cantidad reciclada.
- **FilledButton.icon**: guarda el registro, actualiza las hojas, persiste y avisa con un **SnackBar**.

3.3.18 Impacto sin datos



Figura 21. Impacto sin datos (estado vacío)

Resume el nivel, los kg reciclados y los registros. Sin datos muestra un estado vacío que invita a registrar.

- **MistiBanner**: encabezado con foto e ilustración.
- **CircleAvatar** y **LinearProgressIndicator**: nivel actual y avance al siguiente nivel.
- **Row** con dos **Expanded** (_Estadistica): kg reciclados y número de registros.
- **Table** con **FlexColumnWidth**: resumen por categoría con barras proporcionales.
- **Chip**: camino de niveles (Semilla a Guardián del Misti).

3.3.19 Impacto con registros



Figura 22. Impacto con registros, nivel y tabla de resumen

Con registros guardados se actualizan el nivel, las hojas, la tabla de resumen y el historial con fecha.

- **Table** y **TableRow**: columnas con anchos proporcionales (3, 1.4 y 3) para categoría, kg y barra.
- **Chip** con ícono check_circle y color primaryContainer: niveles alcanzados.
- **Spacer** y **TextButton.icon**: botón Reiniciar alineado a la derecha del título Historial.

3.3.20 Eliminar un registro

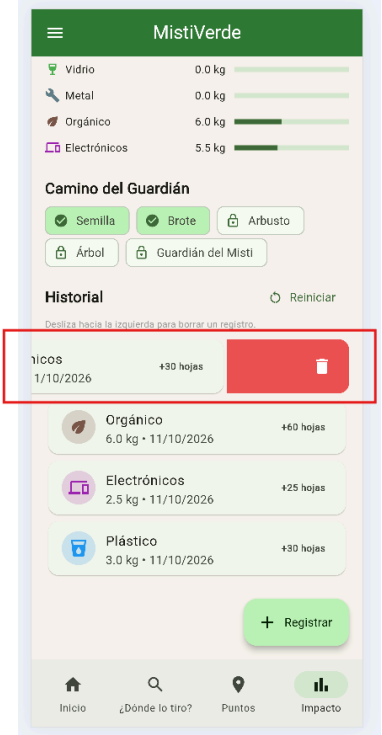


Figura 23. Un registro a medio deslizar, con el fondo rojo

Al deslizar un registro hacia la izquierda se revela un fondo rojo con un ícono de papelera y se elimina al soltar.

- **Dismissible** (endToStart) con key ObjectKey: elimina el elemento al deslizarlo.
- background con **Container** rojo y ícono delete: indica la acción.
- onDismissed llama a onEliminar, que actualiza la lista y guarda los cambios.

3.3.21 Deshacer eliminación



Figura 24. Aviso "Registro eliminado" con el botón "Deshacer"

Tras eliminar, un aviso permite recuperar el registro en su posición original.

- **SnackBar** con **SnackBarAction** (Deshacer): reinserta el registro y vuelve a guardar.
- **ScaffoldMessenger**: reemplaza el aviso anterior con hideCurrentSnackBar.
- behavior floating: el aviso flota sobre la barra inferior.

3.4 Adaptación y temas

La aplicación se adapta al tema del sistema y al tamaño de pantalla. Las capturas siguientes muestran el modo oscuro, la ventana ancha y la orientación horizontal. Cada ajuste se logra con los widgets descritos.

3.4.1 Modo oscuro

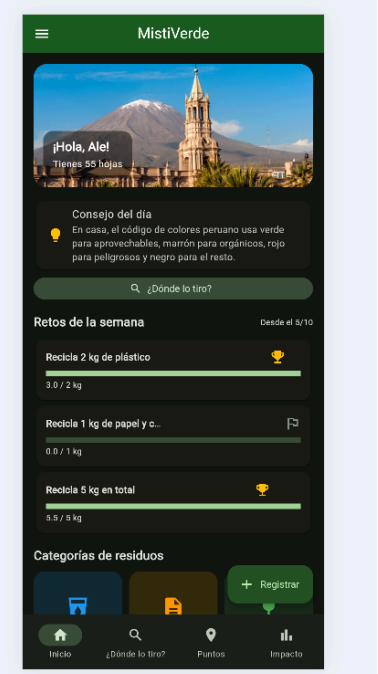


Figura 25. Modo oscuro: Inicio con tema oscuro

El tema oscuro se activa automáticamente cuando el sistema lo usa.

- **ThemeData** y **ColorScheme.fromSeed** con brightness oscuro, definidos en darkTheme.
- **themeMode: ThemeMode.system**: elige claro u oscuro según el sistema.

3.4.2 Pantalla ancha

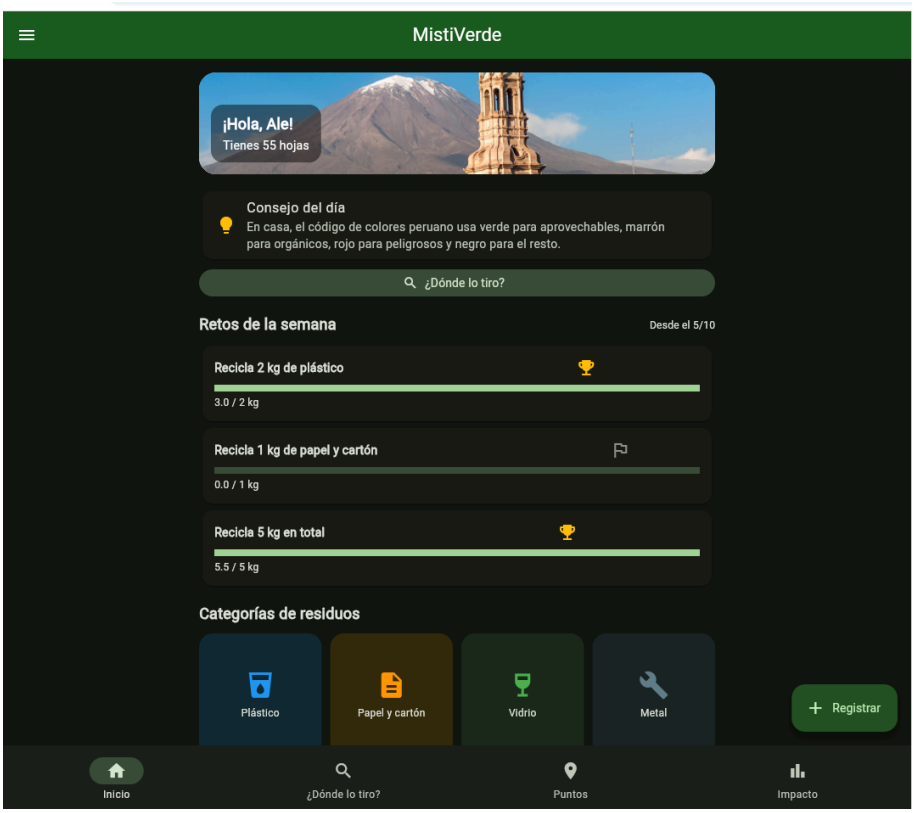


Figura 26. Pantalla ancha: la app en la ventana completa de Chrome

En una ventana grande el contenido no se estira: el ancho se limita a 640 y la cuadrícula pasa a 4 columnas.

- **Center** y **ConstrainedBox** (maxWidth 640): limitan el ancho del contenido.
- **LayoutBuilder**: cambia de 3 a 4 columnas cuando el ancho es de 480 o más.

3.4.3 Horizontal

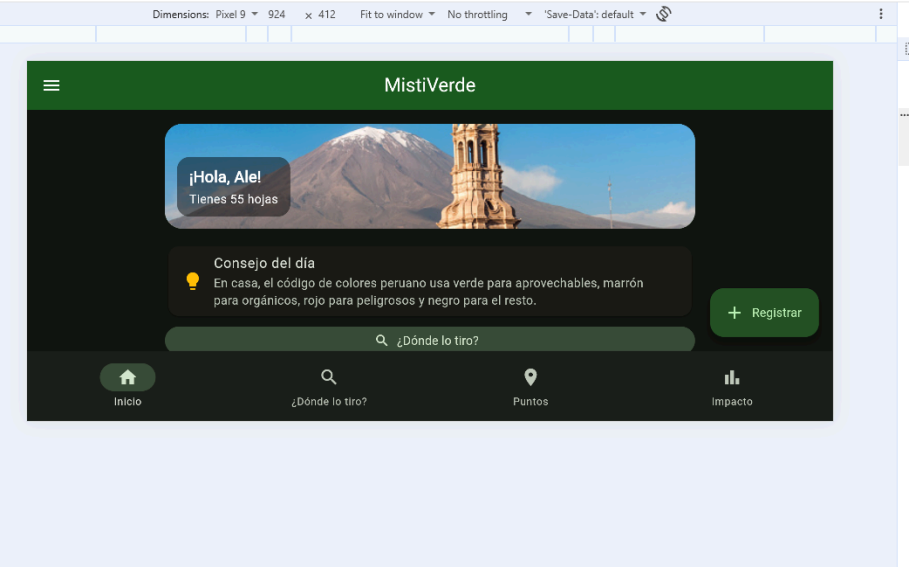


Figura 27. Horizontal: Inicio en orientación horizontal

Al girar el dispositivo, el banner se reduce para dejar más espacio al contenido.

- **MediaQuery** (orientation): la altura del banner baja a 120 en horizontal.

3.5 Resumen de widgets de layout y componentes

La Tabla 6 agrupa los widgets usados según su función en la interfaz.

Tabla 6. Widgets de layout y componentes utilizados

Grupo	Widgets	Función en MistiVerde
Estructura y navegación	Scaffold, AppBar, NavigationBar, Drawer, DrawerHeader, FloatingActionButton.extended, TabBar, TabBarView, PageView, Navigator, PageRouteBuilder	Esqueleto de cada pantalla, cuatro secciones, menú lateral, pestañas Lista y Mapa, onboarding y transiciones.
Disposición	Column, Row, Stack, Positioned, Align, Center, Expanded, Flexible, Spacer, Wrap, SizedBox, Padding, Container, ConstrainedBox, AspectRatio, Table, TableRow	Organizar el contenido en vertical, horizontal y por capas; repartir espacio; limitar el ancho y mantener proporciones.
Adaptación	LayoutBuilder, MediaQuery, GridView.count, SafeArea	Cambiar columnas y alturas según ancho y orientación.
Listas y desplazamiento	ListView, ListView.builder, SingleChildScrollView, CustomScrollView, SliverAppBar, SliverList, SliverPadding, Dismissible	Listas de residuos, puntos e historial; encabezado que se colapsa; deslizar para borrar.
Componentes Material	Card, ListTile, Chip, ChoiceChip, CircleAvatar, FilledButton, TextButton, Slider, LinearProgressIndicator, CircularProgressIndicator, InkWell, Material, Divider, CheckboxListTile	Tarjetas, filas, filtros, botones, selector de kg y barras de progreso.

Entrada de datos	Form, TextFormField, FormField, TextField, TextEditingController	Formulario de perfil validado y búsqueda de residuos.
Retroalimentación	showDialog, AlertDialog, showModalBottomSheet, StatefulBuilder, SnackBar, SnackBarAction, ScaffoldMessenger, showAboutDialog	Diálogos, panel Registrar, avisos y deshacer.
Animación y dibujo	TweenAnimationBuilder, AnimatedSwitcher, AnimatedContainer, FadeTransition, Hero, CustomPaint, CustomPainter, Canvas, Path, ClipRRect, Image.asset	Entradas suaves, cambio de pestaña, ilustración del Misti y fotos con respaldo.
Mapa y datos	FlutterMap, TileLayer, MarkerLayer, Marker, RichAttributionWidget, Clipboard, SharedPreferences	Mapa de puntos, copiar ubicación y guardar datos locales.
Tema	ThemeData, ColorScheme.fromSeed, themeMode	Identidad visual y modo oscuro automático.

3.6 Historial de cambios

Los cambios realizados durante el desarrollo se marcaron como comentarios en el encabezado y en el cuerpo del código (por ejemplo [v4-3]). La Tabla 7 los resume.

Tabla 7. Historial de versiones

Versión	Cambios
v1	Aplicación inicial “EcoArequipa” con 4 pantallas y datos de ejemplo.
v2	Nombre MistiVerde, hojas y niveles, splash, ilustración del Misti, categorías interactivas con Hero, detalle de punto, retos, tema propio y modo oscuro.
v3	Fotos reales desde assets/images/ con ilustración de respaldo.
v4	Colores de tacho según la NTP 900.058:2019, categorías Peligrosos y No aprovechable, persistencia con shared_preferences, retos realmente semanales, puntos de acopio con fuente, layout con ancho máximo, LayoutBuilder, AspectRatio, Table, Spacer y Flexible, filtro en el clasificador y resumen por categoría.
v5	Perfil local con onboarding (PageView), formulario validado (Form) y Drawer; mapa real con flutter_map y OpenStreetMap con TabBar Lista y Mapa; CustomScrollView con SliverAppBar, Dismissible con deshacer, MediaQuery en Inicio y fuentes de puntos de acopio corregidas.

4. APRECIACIÓN

A partir de la experiencia de desarrollar MistiVerde, se señalan dos características a favor y dos desfavorables de Flutter.

4.1 Características a favor

- **Un solo código para varias plataformas.** El mismo main.dart sirve para Android, iOS, web y escritorio. En este trabajo se probó en Chrome sin cambiar el código, lo que reduce el tiempo y el costo de desarrollo.

- **Hot reload y un catálogo amplio de widgets de layout.** Los cambios se ven casi al instante, y widgets como Column, Row, Stack, Wrap, LayoutBuilder o Table, con buena documentación, permitieron armar interfaces completas y adaptables sin escribir código de bajo nivel.

4.2 Características desfavorables

- **Curva de aprendizaje de Dart y de la composición de widgets.** Todo es un widget y las pantallas se construyen anidando muchos, lo que genera código con varios niveles de sangría que cuesta leer y mantener, sobre todo al empezar.
- **Aplicaciones más pesadas y dependencia de paquetes externos.** Las apps de Flutter suelen ocupar más que las nativas y ciertas funciones requieren paquetes de terceros, como shared_preferences para guardar datos y flutter_map para el mapa, que hay que mantener actualizados.

I. CONCLUSIONES

1. Se cumplió el objetivo del laboratorio. Se desarrolló MistiVerde, una aplicación en Flutter que orienta a los arequipeños sobre cómo y dónde reciclar. Sus interfaces atienden cada causa del árbol de problemas: clasificador, puntos de acopio en lista y mapa, registro de reciclaje, hojas y niveles, y retos semanales.
2. Los widgets de layout permitieron armar interfaces adaptables. Column, Row, Stack, Wrap, Expanded, Flexible, Spacer, AspectRatio, Table, LayoutBuilder y MediaQuery bastaron para que la app funcione en modo claro y oscuro, en pantalla ancha y en orientación horizontal. Se logró con el mismo código y sin recurrir a código de bajo nivel.
3. Los componentes propios de Flutter cubrieron la interacción de la app. Entre ellos están Form con validación, PageView, Drawer, TabBar, CustomScrollView con SliverAppBar, Dismissible con deshacer, diálogos, paneles inferiores y animaciones como Hero. Además, shared_preferences y flutter_map resolvieron la persistencia local y el mapa con OpenStreetMap.
4. Un solo código base sirvió para varias plataformas. La app se probó en Chrome sin modificar el código. Esto confirma la ventaja principal de un framework multiplataforma: menos tiempo y costo de desarrollo, con una interfaz coherente en cada dispositivo.
5. Hay limitaciones que se declararon con honestidad. El perfil es local y no hay autenticación real. Los kilogramos son autodeclarados y los puntos de acopio vienen de fuentes públicas sin horarios verificados. Los marcadores del mapa son referenciales y no se midió el rendimiento. Como trabajo futuro se podrían agregar GPS, direcciones verificadas con la Municipalidad y una base de datos en la nube.
6. La experiencia con Flutter dejó ventajas y desventajas. Hot reload y la documentación facilitaron el desarrollo. En contra, el anidamiento de widgets hace el código difícil de leer, y la dependencia de paquetes externos exige mantenerlos actualizados.

REFERENCIAS Y BIBLIOGRAFÍA

[1] Flutter, "Layouts in Flutter," Flutter Documentation. [Online]. Available: https://docs.flutter.dev/ui/layout (accessed Oct. 11, 2026).

[2] Flutter, "Widget catalog," Flutter Documentation. [Online]. Available: https://docs.flutter.dev/ui/widgets (accessed Oct. 11, 2026).

[3] Flutter, "Flutter API reference." [Online]. Available: https://api.flutter.dev (accessed Oct. 11, 2026).

[4] Dart, "Dart documentation." [Online]. Available: https://dart.dev/guides (accessed Oct. 11, 2026).

[5] pub.dev, "shared_preferences." [Online]. Available: https://pub.dev/packages/shared_preferences (accessed Oct. 11, 2026).

[6] pub.dev, "flutter_map." [Online]. Available: https://pub.dev/packages/flutter_map (accessed Oct. 11, 2026).

[7] pub.dev, "latlong2." [Online]. Available: https://pub.dev/packages/latlong2 (accessed Oct. 11, 2026).

[8] Google, "Material Design 3." [Online]. Available: https://m3.material.io (accessed Oct. 11, 2026).

[9] OpenStreetMap contributors, "Copyright and license." [Online]. Available: https://www.openstreetmap.org/copyright (accessed Oct. 11, 2026).

[10] Andina, "Inacal invoca a la ciudadanía aplicar norma peruana para almacenar residuos sólidos (NTP 900.058:2019)." [Online]. Available: https://andina.pe/ingles/noticia-inacal-invoca-a-ciudadania-aplicar-norma-peruana-para-almacenar-residuos-solidos-749897.aspx (accessed Oct. 11, 2026).

[11] Municipalidad Provincial de Arequipa, Arequipa Recicla [Mobile app]. Google Play.

[12] Diario Correo Arequipa, "Dónde depositar tus residuos electrónicos en Arequipa: conoce aquí los puntos de acopio," 2023. [Online]. Available: https://diariocorreo.pe/edicion/arequipa/donde-depositar-tus-residuos-electronicos-en-arequipa-conoce-aqui-los-puntos-de-acopio-video-noticia/ (accessed Oct. 11, 2026).

[13] Recolecc, "Puntos de acopio." [Online]. Available: https://recolecc.com.pe/puntos-de-acopio/ (accessed Oct. 11, 2026).

[14] Energiminas, "SIDERPERÚ inaugura en Arequipa el centro para reciclaje de chatarra más moderno del país," Oct. 21, 2024. [Online]. Available: https://energiminas.com/2024/10/21/siderperu-inaugura-en-arequipa-el-centro-para-reciclaje-de-chatarra-mas-moderno-del-pais/ (accessed Oct. 11, 2026).

[15] Revista Economía, "Ubicación del Centro de Reciclaje de Chatarra de SIDERPERÚ en la Asociación Bellavista, Yura," 2024.

[16] Wikipedia, "Distritos de Cerro Colorado y Yura (coordenadas de referencia)." [Online]. Available: https://es.wikipedia.org (accessed Oct. 11, 2026).